



OpenClaw 深度解析：从 0 到入门 AI Harness 工程

背景假设：C++/Python 编程基础，熟悉 LLM/VLM transformers 架构及 model serving，了解 system prompt/user prompt，有部署 OpenClaw + 飞书 channel 的实际经验，无 TypeScript 经验。

零、OpenClaw 本身就是用 Claude 辅助编写的

从 git 历史的 `Co-Authored-By` 统计可以看出：

Co-Author	commit 次数
Claude Opus 4.6	316
人类贡献者 Tak Hoffman	280
人类贡献者 gumadeiras	263
Claude Opus 4.5	137
Claude Opus 4.6 (1M context)	87
Claude （未标版本）	55

Claude 各版本合计约 **595** 次 co-authored commit，是单个最大的"贡献者"。

但注意标记是 **Co-Authored-By**（共同作者），不是独立作者——人类开发者使用 Claude Code 辅助写代码，人类做审查和决策，Claude 做实现。

这也解释了为什么仓库根目录的 `CLAUDE.md` 写得如此详细（400+ 行规范）——它本质上就是给 **Claude Code** 的项目级指令，让 AI 在这个仓库工作时遵守架构边界和编码规范。`CLAUDE.md` 是项目维护者提交在 git 中的，不是 AI 自动生成的。

命名说明（来自 [CLAUDE.md L12](#)）：对外统一使用 "plugin" 一词（docs、UI、changelog），但 `extensions/` 目录名是历史遗留的内部包布局，为了避免全仓库重命名

的大量无意义 diff 而保留。所以 **extension = plugin**，只是目录名没改。

一、什么是 AI Harness?

AI Harness 是 **LLM** 的运行外壳——类似 model serving (vLLM/TGI)，但 harness 关注的不是推理吞吐，而是：

- **Prompt 编排**：如何组装 system prompt → user message → tool results 的完整上下文
- **工具调用循环**：LLM 输出 tool_call → 执行 → 结果回填 → 再次调用 LLM
- **多通道接入**：Telegram/Discord/飞书/Web 等渠道统一接入同一个 Agent
- **插件扩展**：通过注册机制让第三方扩展 provider、channel、tool、hook

用 C++ 类比：harness 就像一个 **event loop + plugin framework**，LLM API 是其中的一个异步 I/O 源。

二、项目总体架构

```
openclaw/
├─ src/                # 核心平台 (TypeScript)
│  ├─ agents/          # Agent 运行时 + system prompt 构建
│  ├─ gateway/         # WebSocket 控制面 (类似 gRPC server)
│  ├─ channels/        # 核心消息通道实现
│  ├─ routing/         # 消息路由 (channel → agent → session)
│  ├─ plugins/         # 插件发现、加载、注册表
│  ├─ plugin-sdk/      # 插件公开 SDK (70+ 子路径导出)
│  ├─ hooks/           # 内部事件钩子系统
│  ├─ cli/             # CLI 入口
│  ├─ commands/        # 命令实现
│  ├─ config/          # 配置管理
│  ├─ mcp/             # Model Context Protocol 支持
│  └─ ...              # media, tts, web-search, cron 等
├─ extensions/         # 107 个 bundled 插件 (provider/channel/capability)
├─ packages/           # 共享包 (plugin-sdk, contracts)
├─ skills/             # 53 个 bundled skills (agent 能力)
├─ apps/               # 原生应用 (macOS/iOS/Android)
├─ ui/                 # Web UI
└─ docs/               # 文档
```

核心设计原则

原则	说明	C++/Python 类比
Manifest-first	插件通过 JSON 清单声明能力，不执行代码即可校验	类似 <code>.so</code> 的 metadata section
Registry pattern	运行时通过注册表查找 provider/channel/tool	类似 factory pattern + service locator
Lazy loading	重模块按需动态 import，不拖慢启动	类似 <code>dlopen()</code>
Cache boundary	System prompt 分为静态区（可缓存）和动态区	类似 KV cache 的 prefix caching
Fail-open / Fail-	Hook 默认 fail-open，tool 审批 hook	类似中间件的错误策略

原则	说明	C++/Python 类比
closed	fail-closed	

三、端到端消息流（最核心的流程）

以飞书用户发一条消息到收到回复的完整路径为例：

飞书 webhook → Gateway (WebSocket server)

↓

chat.ts (验证、解析附件)

↓

resolve-route.ts (路由决策)

└ 匹配 channel + peer → agentId

└ 生成 sessionId (如 "feishu:direct:user123")

↓

dispatchInboundMessage() (消息分发)

↓

SessionManager 加载会话历史 (JSONL 格式)

↓

buildAgentSystemPrompt()

组装完整 system prompt (见第四节)

↓

runEmbeddedPiAgent() (主运行循环)

while(true)

1. 选择 model + auth profile

2. 调用 LLM API (stream)

3. 如果返回 tool_call:

→ 执行 tool

→ 结果写入 session

→ 继续循环

4. 如果 stop_reason=end_turn:

→ 跳出循环

5. 错误处理:

→ 超时 → compaction 压缩

→ 限流 → 切换 auth profile

→ 溢出 → 截断历史

↓

Reply formatting (channel 适配)

↓

message.ts → 飞书 API → 用户看到回复

关键洞察：这个 `while(true)` 循环就是 AI Agent 的核心——它不是一次 API 调用，而是一个 **ReAct loop** (Reasoning + Acting)，LLM 可以连续调用多个工具后再给出最终回答。

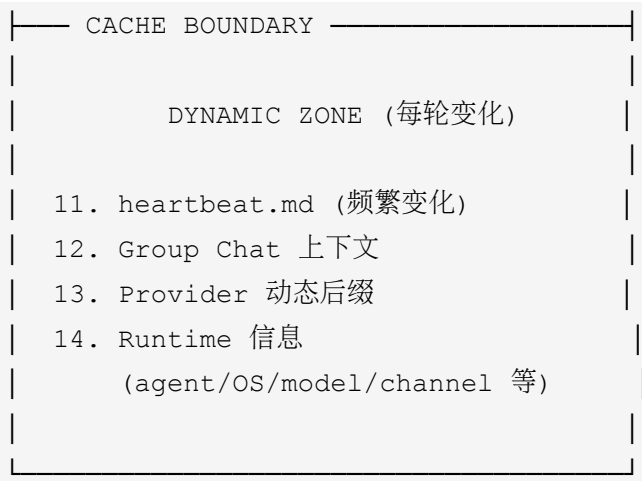
四、Prompt 编排架构 (Harness 的灵魂)

OpenClaw 的 system prompt 不是一个静态字符串，而是一个 动态组装的多层结构。

4.1 System Prompt 构建流程

核心函数: `src/agents/system-prompt.ts` 中的 `buildAgentSystemPrompt()`

STATIC ZONE (可缓存)
1. Basic Identity
"You are a personal assistant operating inside OpenClaw."
2. Tooling 指导
工具使用规范、subagent 规范
3. Safety 规则
安全边界、safeguards
4. CLI Reference
gateway 命令、自更新策略
5. Skills 列表 (XML 格式)
<available_skills>
<skill>
<name>coding-agent</name>
<description>...</description>
</skill>
</available_skills>
6. Memory 指导
7. Model Aliases
8. Workspace 信息
9. 上下文文件 (确定性排序)
agents.md → soul.md →
identity.md → user.md →
tools.md → bootstrap.md →
memory.md
10. Plugin prepend/append context
(via before_prompt_build hook)



4.2 Cache Boundary 的意义

这是 harness engineering 的精华之一。Anthropic API 支持 **prompt caching**：如果连续请求的 system prompt 前缀字节完全一致，后端可以复用已计算的 KV cache。

OpenClaw 将 prompt 分为：

- **Static zone**：identity、rules、skills、context files → 跨轮次不变
- **Dynamic zone**：heartbeat、group context、runtime info → 每轮可能变

这意味着多轮对话中，API 只需要处理 dynamic zone 的增量，大幅降低 TTFT（Time To First Token）和成本。

4.3 Prompt 的三种模式

```
type PromptMode = "full" | "minimal" | "none";
```

模式	用途	包含的 section
full	主 Agent	最多 26 个 section（约 16 个始终存在，10 个条件性）
minimal	Subagent（子任务）	仅 Tooling + Workspace + Runtime
none	极简场景	仅 identity 一行

这类似于 serving 中的不同 prefix template——主 agent 需要完整上下文，而 subagent 只需要

工具和工作目录信息。

full 模式下的全部 `##` section（按出现顺序，源码位于 `src/agents/system-prompt.ts`）：

#	Section	条件	说明
1	Tooling	始终	工具使用规范、cron/subagent/ACP 指导
2	Tool Call Style	始终（可被 provider 覆盖）	工具调用叙述策略、审批指导
3	Execution Bias	始终（可被 provider 覆盖）	执行偏好（主动执行 vs 确认后执行）
4	Safety	始终	安全边界、禁止自我复制/扩权
5	OpenClaw CLI Quick Reference	始终	gateway 子命令速查
6	Skills (mandatory)	有 skills 时	<code><available_skills></code> XML 列表
7	Memory	有 memory 插件时	记忆插件使用指导
8	OpenClaw Self-Update	有 gateway 时	config/update 操作规范
9	Model Aliases	有配置时	模型别名映射表
10	Workspace	始终	工作目录路径 + 文件操作指导
11	Documentation	始终	文档路径、社区链接
12	Sandbox	sandbox 启用时	Docker 沙盒策略、路径映射
13	Authorized Senders	有 owner 配置时	授权用户身份信息
14	Current Date & Time	有 timezone 时	时区 + session_status 指导
15	Workspace Files (injected)	始终	声明下方将注入用户上下文文件
16	Reply Tags	始终	通道原生回复/引用语法

#	Section	条件	说明
17	Messaging	始终	会话路由、消息工具使用、subagent 编排
18	Voice (TTS)	有 TTS 配置时	语音合成提示
19	Reactions	有 reaction 配置时	emoji 反应策略 (minimal/ extensive)
20	Reasoning Format	有 reasoning 配置时	think/final 标签格式
21	Silent Replies	始终	<code>__SILENT__</code> token 使用规则
—	== CACHE BOUNDARY ==	—	静态区/动态区分界线
22	Project Context	有 context files 时	稳定的用户上下文文件内容
23	Dynamic Project Context	有动态文件时	heartbeat.md 等频繁变化的文件
24	Group Chat Context	有 extra prompt 时	群聊/session 特定上下文
25	Heartbeats	有 heartbeat 时	心跳轮询指导
26	Runtime	始终	agent/OS/model/channel/ capabilities 信息

其中约 **16** 个始终存在 (1-5, 10-11, 15-17, 21, 26 + Project Context/Runtime)，约 **10** 个条件性 (6-9, 12-14, 18-20, 24-25)，具体取决于用户配置和运行环境。

4.4 上下文文件注入

用户可以在工作目录 (如 `~/.openclaw/`) 放置**固定文件名**的 markdown 文件来定制 Agent 行为。只有以下 8 个文件名会被扫描加载 (硬编码在 `src/agents/workspace.ts:26-34`)，任意其他 `.md` 文件不会被注入 prompt：

磁盘文件名	排序优先级	作用
AGENTS.md	10	多 Agent 路由规则
SOUL.md	20	Agent 人设/语气（触发特殊 persona 指导）
IDENTITY.md	30	自定义身份
USER.md	40	用户/owner 上下文
TOOLS.md	50	自定义工具使用指导
BOOTSTRAP.md	60	额外引导
MEMORY.md	70	记忆插件指导（也接受小写 memory.md）
HEARTBEAT.md	dynamic	频繁变化的信息（放在 cache boundary 之下）

扫描逻辑：逐个拼出这 8 个固定路径 → 尝试读取 → 文件存在就加载内容，不存在就跳过。

排序是确定性的（system-prompt.ts 中用小写化后的 basename 匹配 CONTEXT_FILE_ORDER Map），确保 prompt caching 的字节稳定性。

五、插件系统（Extension/Plugin Architecture）

5.1 插件生命周期

Discovery → Manifest Validation → Enablement → Loading → Registration → Execution

1) Discovery（发现）

src/plugins/discovery.ts 从多个来源扫描：

- extensions/ 目录（bundled 插件，107 个）
- ~/.openclaw/extensions/（用户安装的）
- plugins.load.paths（配置指定的）

安全检查：路径逃逸检测、权限检查、可疑 ownership 检测。

2) Manifest Validation（清单校验）

每个插件目录下有 `openclaw.plugin.json`：

```
{
  "id": "feishu",
  "channels": ["feishu"],
  "channelEnvVars": {
    "feishu": ["FEISHU_APP_ID", "FEISHU_APP_SECRET", "FEISHU_ENCRYPT_KEY"]
  },
  "configSchema": { "type": "object", "additionalProperties": false }
}
```

Manifest-first 原则：不执行任何代码就能知道插件提供什么能力、需要什么配置。这类似 Python 的 `entry_points` 或 C++ 的 `plugin metadata`。

3) Loading（加载）

通过 Jiti（运行时 TypeScript 转译器）动态加载插件入口：

```
// 典型的插件入口文件
import { definePluginEntry } from "openclaw/plugin-sdk/plugin-entry";

export default definePluginEntry({
  id: "my-plugin",
  name: "My Plugin",
  register(api) {
    // 注册能力
    api.registerTool({ ... });
    api.registerChannel({ ... });
    api.registerProvider({ ... });
  },
});
```

4) Registration（注册）

`register(api)` 被调用时，插件通过 `OpenClawPluginApi` 注册各种能力：

```
api.registerProvider()           → 模型推理 provider (如 Anthropic、OpenAI)
api.registerChannel()           → 消息通道 (如飞书、Telegram)
api.registerTool()               → Agent 工具 (如 web_search、code_exec)
api.registerHook()               → 生命周期钩子
api.registerSpeechProvider()     → TTS/STT
api.registerHttpRoute()         → HTTP 端点
api.registerService()           → 后台服务
// ... 40+ 种注册方法
```

所有注册的能力汇入中央 `PluginRegistry`，运行时通过 `registry` 查找。

5.2 插件分类

OpenClaw 的 107 个 `bundled` 插件覆盖：

类别	数量	示例
AI Provider	35+	anthropic, openai, deepseek, qwen, ollama, vllm
消息通道	25+	telegram, discord, slack, feishu, matrix, whatsapp
搜索引擎	5+	brave, duckduckgo, exa, tavily
媒体处理	10+	deepgram(STT), elevenlabs(TTS), runway(video)
开发工具	5+	github, kilocode
记忆系统	3	memory-core, memory-lancedb, memory-wiki

5.3 架构边界规则

OpenClaw 代码规范的核心——严格的 `import boundary`：

extension 代码 —只能import—→ openclaw/plugin-sdk/*
(70+ 子路径, 按需导入)

core 代码 —不能import—→ extension 内部文件
(不能 import extensions/feishu/src/...)

plugin-sdk —作为契约层—→ 连接 core 和 extension

用 C++ 类比: plugin-sdk 就像一个 .h 头文件定义的 ABI, core 和 extension 都只依赖这个接口, 不直接耦合。

六、Hook 机制

OpenClaw 有两套 hook 系统:

6.1 Plugin Hooks (主要: Agent 生命周期)

定义在 `src/plugins/types.ts`, 共 27 种 hook, 按阶段分:

Prompt 修改类

<code>before_model_resolve</code>	→ 覆盖 model/provider 选择
<code>before_prompt_build</code>	→ 修改 system prompt (注入上下文)
<code>before_agent_start</code>	→ 遗留兼容 hook (组合两个阶段)

`before_prompt_build` 最重要——插件通过它注入额外上下文到 system prompt:

```
// Hook 返回值
type PluginHookBeforePromptBuildResult = {
  systemPrompt?: string;           // 替换整个 system prompt
  prependContext?: string;         // 动态前置 (每轮变化, 不缓存)
  prependSystemContext?: string;   // 静态前置 (放在 cache boundary 之上)
  appendSystemContext?: string;    // 静态后置 (放在 cache boundary 之上)
};
```

注意 `prependSystemContext` VS `prependContext` 的区别——前者放在 `cache boundary` 之上（可缓存），后者在之下（每轮重新计算）。这是 `harness` 工程中对 `prompt caching` 的精细控制。

Tool 执行类

```
before_tool_call → 工具执行前（fail-closed: 出错则阻止执行）
after_tool_call  → 工具执行后（观察/记录）
```

消息生命周期

```
message_received    → 收到消息
message_sending     → 发送前（可修改内容）
message_sent        → 发送后
message_transcribed → 音频转文字完成
```

Agent/Session 生命周期

```
session_start / session_end
subagent_spawning / subagent_spawned / subagent_ended
agent_end
before_compaction / after_compaction → 会话压缩
gateway_start / gateway_stop
```

Hook 执行模型

```
// 注册
ctx.on("before_tool_call", handler, { priority: 10 });

// 执行顺序: priority 高的先执行
// 错误策略:
//   默认 fail-open（出错继续）
//   before_tool_call 是 fail-closed（出错阻止）
```

6.2 Internal Hooks（事件广播）

`src/hooks/internal-hooks.ts` 使用 `Symbol.for()` 实现全局单例，事件类型：

```
"command" → CLI 命令执行
"session" → Session 状态变更
"agent" → Agent bootstrap / compaction
"gateway" → 启停
"message" → 收发/转录/预处理
```

用途：内部组件间的松耦合通信，类似 C++ 的 observer pattern 或 Python 的 signal/slot。

七、Skills 系统

Skills 是预定义的 **Agent** 能力包，当前有 53 个 bundled skill。

7.1 Skill 的本质

每个 skill 是一个目录，包含 `SKILL.md`（Markdown 格式的指令），当 Agent 需要时读取并注入到上下文中。

```
skills/
├─ coding-agent/SKILL.md → 编程能力
├─ github/SKILL.md      → GitHub 操作
├─ slack/SKILL.md       → Slack 集成
├─ weather/SKILL.md     → 天气查询
└─ ...
```

7.2 Skill 发现与注入

`src/agents/skills/workspace.ts` 中的 `resolveSkillsPromptForRun()`：

1. 发现：从 bundled + managed + workspace + personal 目录加载
2. 过滤：根据配置、可见性、agent 约束过滤
3. 限制：最多 150 个 skill，总 prompt ≤ 30KB

4. 格式化：转为 XML 注入到 system prompt

```
<available_skills>
  <skill>
    <name>coding-agent</name>
    <description>Full-stack coding assistance</description>
    <location>~/.openclaw/skills/coding-agent/SKILL.md</location>
  </skill>
</available_skills>
```

Agent 在需要时会先读取 `SKILL.md` 文件内容，然后按照指令执行。

八、Provider 系统（模型调用）

8.1 Provider Plugin 架构

每个 LLM provider（Anthropic、OpenAI、DeepSeek 等）是一个插件，通过 `api.registerProvider()` 注册，提供 **44** 个有序 **hook**：

<code>catalog</code>	→ 发布模型目录
<code>normalizeModelId</code>	→ 模型 ID 别名解析
<code>normalizeTransport</code>	→ 传输层配置
<code>buildRequest</code>	→ 构建 API 请求
<code>parseResponse</code>	→ 解析 API 响应
<code>handleStream</code>	→ 处理流式响应
<code>onModelSelected</code>	→ 模型选中后的副作用
...	(共 44 个)

这类似 vLLM 中 model runner 的生命周期，但抽象层更高。

8.2 Auth Profile 与 Failover

运行循环中内置了 auth 轮换机制：

- 多个 API key profile 可配置

- 遇到 rate limit → 自动切换到下一个 profile
- 遇到 auth 失败 → 标记 profile 并冷却
- 所有 profile 耗尽 → 触发 model fallback（切换到备选模型）

九、Gateway 架构

Gateway 是 OpenClaw 的中央协调器，运行在 `localhost:18789`：



Protocol 定义在 `src/gateway/protocol/schema.ts`，是一个 版本化的 **wire contract**——类似 gRPC 的 proto 文件，变更需要向后兼容。

十、关键设计模式总结

作为想理解 harness engineering 的学习者，这些是最值得学习的模式：

1. Prompt Cache Stability (Prompt 缓存稳定性)

问题：每次组装 prompt 的字节顺序不同，会导致 API 端的 prefix cache miss。

方案：确定性排序所有来源（plugins、tools、skills、files），静态/动态内容用 cache boundary 隔开。

2. Manifest-First Plugin Loading

问题：加载插件代码很慢，且可能有副作用。

方案：先解析 JSON manifest 获得元数据，只在真正需要时才执行 `register()`。

3. Hook Priority + Error Policy

问题：多个插件的 hook 如何排序？hook 出错是否阻断流程？

方案：数值 priority（高优先），默认 fail-open，安全关键路径（tool 执行）fail-closed。

4. ReAct Loop with Failover

问题：Agent 执行可能超时、限流、上下文溢出。

方案：`while(true)` 循环 + compaction（压缩历史）+ auth 轮换 + model fallback。

5. Lazy Loading + Import Boundary

问题：107 个插件全部启动时加载太慢。

方案：hot path（channel.ts、shared.ts）轻量静态导入，heavy path（send、monitor）动态 `await import()`。

十一、动手路径建议

推荐阅读顺序

1. 读 **system prompt** 构建：`src/agents/system-prompt.ts` — 理解 harness 的最佳入口

2. 读主运行循环: `src/agents/pi-embedded-runner/run.ts` — 理解 ReAct loop
3. 读一个简单插件: `extensions/brave/` (搜索插件, 结构简单)
4. 读路由逻辑: `src/routing/resolve-route.ts` — 消息如何找到对的 Agent
5. 读飞书通道: `extensions/feishu/` — 你已经部署过, 有实际体感

TypeScript 对照表 (给 C++/Python 背景)

TypeScript	C++	Python
<code>interface</code>	纯虚类	<code>Protocol</code> / <code>ABC</code>
<code>type X = A B</code>	<code>std::variant<A,B></code>	<code>Union[A, B]</code>
<code>async/await</code>	coroutines	<code>asyncio</code>
<code>import { X } from "y"</code>	<code>#include</code> + namespace	<code>from y import X</code>
<code>Record<K, V></code>	<code>std::map<K,V></code>	<code>Dict[K, V]</code>
<code>z.object({...})</code> (Zod)	—	<code>pydantic.BaseModel</code>